

Pro Weather API Clone

Project Documentation & System Architecture

1. Executive Summary

This project is a high-performance, modular FastAPI implementation designed to clone the core functionality of an Open Weather API. It leverages the **Open-Meteo** ecosystem for geocoding and meteorological data. The architecture follows a "Clean Code" approach, separating business logic, data validation, and persistence layers.

Core Technical Stack

FastAPI

Pydantic v2

HTTPX (Async)

Uvicorn

JSON Persistence

2. Folder Hierarchy (Architecture)

```
weather-api-clone/
├── data/
│   └── data.json          # Query history storage
├── src/
│   ├── __init__.py
│   ├── main.py            # API Endpoints
│   ├── models.py          # Pydantic Schemas
│   ├── services.py        # External API Integration
│   └── database.py         # File I/O Logic
├── requirements.txt
└── README.md
```

3. Data Validation (Pydantic)

The system uses Pydantic `BaseModel` to enforce strict data types. This ensures that the `data.json` file remains consistent and the API provides accurate responses.

Field	Type	Validation Rule
city	String	Min length: 1
temperature	Float	Range: -90°C to 60°C
windspeed	Float	Value >= 0
date_of_request	Date	ISO Format

4. Implementation Guide

Execution Command

Run the application from the root folder to maintain package integrity:

```
uvicorn src.main:app --reload
```

Endpoint: Get Weather

GET `/weather/{city}`

The internal workflow handles geocoding first, then fetches current weather, validates the combined result, and performs a deduplicated log write.

5. Automated Cleanup Logic

To prevent storage bloat, the system implements a `save_and_clean` function. When a city is searched, the system checks the existing history in `data.json`. If the city already exists, the entry is updated with the most recent data

(temperature, windspeed, and timestamp), ensuring the file contains only **unique city records**.

Generated for: Weather API Clone Project © 2024